Assignment No. - 10

Title: Aggregation and Indexing in MongoDB.

Problem Statement:

Design and execute MongoDB queries using aggregation and indexing techniques with Suitable examples.

Objective:

- To learn concept of aggregation and indexing.
- To implement aggregation and indexing techniques in MongoDB.

Theory:

MongoDB is an open-source document database and leading NoSQLdatabase. MongoDB is written in C++. This tutorial will give you great understanding on MongoDB concepts needed to create and deploy a highly scalable and performance- oriented database.

Aggregations operations process data records and return computed results. Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result. In SQL count (*) and with group by isan equivalent of MongoDB aggregation.
The aggregate () Method for the aggregation in MongoDB, you should use aggregate ()method. Basic syntax of aggregate () method is as follows:

>db.COLLECTION_NAME.**aggregate**(AGGREGATE_OP
ERATION)

EXAMPLE

In the collection you have the following data:
{
_id:
ObjectId(7df78ad8902c)title:
'MongoDB

Overview',
description: 'MongoDB is no sql database',by_user: 'tutorials point',
url: 'http://www.tutorialspoint.com', tags: ['mongodb', 'database', 'NoSQL'],likes: 100
},

{
_id: ObjectId(7df78ad8902d)
title: 'NoSQL Overview',
description: 'No sql database is very fast',by_user: 'tutorials point',
url: 'http://www.tutorialspoint.com', tags: ['mongodb', 'database', 'NoSQL'],likes: 10
},

{
_id: ObjectId(7df78ad8902e)title: 'Neo4j Overview',description: 'Neo4j is no sql database',by_user: 'Neo4j', url: 'http://www.neo4j.com',

tags: ['neo4j', 'database', 'NoSQL'],likes: 750
},

Now from the above collection, if you want to display a list stating how many tutorialsare written by each user,then you will use the following aggregate() method:

> DB.MYCOL.AGGREGATE([{$GROUP : {_ID : "$BY_USER", NUM_TUTORIAL : {$SUM :1}}}])

{
"result" : [
{
"_id" : "tutorials point", "num_tutorial" : 2
},

{
"_id" : "Neo4j","num_tutorial" : 1
}],

"ok" : 1
}>

Sql equivalent query for the above use case will be select by_user, count(*) frommy col group by_user Pipeline Concept

In UNIX command, shell pipeline means the possibility to execute an operation onsome input and use the output as the input for the next command and so on.

MongoDB also supports same concept in aggregation framework. There is a set ofpossible stages and each of those is taken as a set of documents as an input and produces a resulting set of documents (or the final resulting JSON document at theend of the pipeline). This can then in turn be used for the next stage and so on.

Following are the possible stages in aggregation framework:

$project: Used to select some specific fields from a collection.

$match: This is a filtering operation and thus this can reduce the amount ofdocuments that are given as input to the next stage.

$group: This does the actual aggregation as discussed above.

$sort: Sorts the documents.

$skip: With this, it is possible to skip forward in the list of documents for a given amount of documents.

$limit: This limits the amount of documents to look at, by the given number starting from the current positions. $unwind: This is used to unwind document that are using arrays. When using an array, thedata is kind of pre-joined and this operation will be undone with this to have individual documents again. Thus with this stage we will increase the amount of documents for the nextstage

Expression	Description	Example
$sum	Sums up the defined value from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : "$likes"}}}])
$avg	Calculates the average of all given values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$avg : "$likes"}}}])
$min	Gets the minimum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$min : "$likes"}}}])
$max	Gets the maximum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$max : "$likes"}}}])
$push	Inserts the value to an array in the resulting document.	db.mycol.aggregate([{$group : {_id : "$by_user", url : {$push: "$url"}}}])
$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.	db.mycol.aggregate([{$group : {_id : "$by_user", url : {$addToSet : "$url"}}}])
$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "$sort"-stage.	db.mycol.aggregate([{$group : {_id : "$by_user", first_url : {$first : "$url"}}}])
$last	Gets the last document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "$sort"-stage.	db.mycol.aggregate([{$group : {_id : "$by_user", last_url : {$last : "$url"}}}])

Indexes: support the efficient resolution of queries. Without indexes, MongoDB must scan every document of a collection to select those documents that match the query statement This scan is highlyinefficient and require MongoDB to process a large volume of data.
Indexes are special data structures that store a small portion of the data set in an easy -to-traverse form. The index stores the value of a specific field or set of fields, ordered by the value of the field as specified in the index.

THE ENSUREINDEX() METHOD

To create an index you need to use ensureIndex() method of MongoDB. The basic syntaxofensureIndex() method is as follows().
>db.COLLECTION_NAME.ensureIndex({KEY:1})

Here key is the name of the file on which you want to create index and 1 is for ascendingorder. Tocreate index in descending order you need to use -1.

EXAMPLE

>db.mycol.ensureIndex({"title":1})

In **ensureIndex()** method you can pass multiple fields, to create index on multiple fields.

>db.mycol.ensureIndex({"title":1,"description":-1})

ensureIndex() method also accepts list of options (which are optional). Following is the list:

Parameter	Type	Description
background	Boolean	Builds the index in the background so that building an index does not block other database activities. Specify true to build in the background. The default value is **false**.
unique	Boolean	Creates a unique index so that the collection will not accept insertion of documents where the index key or keys match an existing value in the index. Specify true to create a unique index. The default value is **false**.

name	String	The name of the index. If unspecified, MongoDB generates an index name by concatenating the names of the indexed fields and the sort order.
dropDups	Boolean	Creates a unique index on a field that may have duplicates. MongoDB indexes only the first occurrence of a key and removes all documents from the collection that contain subsequent occurrences of that key. Specify true to create unique index. The default value is **false**.
sparse	Boolean	If true, the index only references documents with the specified field. These indexes use less space but behave differently in some situations (particularly sorts). The default value is **false**.
expireAfterSeconds	Integer	Specifies a value, in seconds, as a TTL to control how long MongoDB retains documents in this collection.
v	Index Version	The index version number. The default index version depends on the version of MongoDB running when creating the index.
weights	Document	The weight is a number ranging from 1 to 99,999 and denotes the significance of the field relative to the other indexed fields in terms of the score.

weights	Document	The weight is a number ranging from 1 to 99,999 and denotes the significance of the field relative to the other indexed fields in terms of the score.

default_language	String	For a text index, the language that determines the list of stop words and the rules for the stemmer and tokenizer. The default value is **english**.
language_override	String	For a text index, specify the name of the field in the document that contains, the language to override the default language. The default value is language.

Conclusion: - Thus we have studied and implemented aggregation function & indexing function.

FAQ:

1. Enlist various aggregation operations.
2. Explain MIN function with example.
3. Explain PUSH function with example.
4. Explain SUM & AVG function with example.